Algoritmo e Laboratório de Programação II

Aula 10 – Operações com Arquivos – Parte 1 (Nível Avançado)

Roteiro

- Arquivos
- Leitura e gravação em alto-nível
- Abrindo arquivos com fopen()
- Gravar e ler um caractere por vez
- A função feof()
- Fim de arquivo e a constante EOF
- Cuidados ao abrir o arquivo
- Ler e gravar linha a linha
- Ler e gravar dados formatados
- Modos de abertura de arquivos
- Modo texto e modo binário
- Ler e gravar blocos de bytes
- Os ponteiros de arquivos stdin, stdout e stderr

Introdução

- Nesta aula, você aprenderá os recursos da linguagem C para operações de leitura e gravação em discos.
- Começaremos com uma breve descrição das funções usadas para executar essas tarefas, conhecidas como funções de io.
- Em seguida, mostraremos como elas podem ser aplicadas a programas que necessitam manipular arquivos, com exemplo de diferentes situações.

Roteiro

- Arquivos
- Leitura e gravação em alto-nível
- Abrindo arquivos com fopen()
- Gravar e ler um caractere por vez
- A função feof()
- Fim de arquivo e a constante EOF
- Cuidados ao abrir o arquivo
- Ler e gravar linha a linha
- Ler e gravar dados formatados
- Modos de abertura de arquivos
- Modo texto e modo binário
- Ler e gravar blocos de bytes
- Os ponteiros de arquivos stdin, stdout e stderr

Arquivos

- A palavra arquivo será usada para indicar um ‘fluxo de bytes’ (stream).
- Todo lugar que tem capacidade para receber bytes da memória do computador ou transferi-los para ela recebe o nome de arquivo.
- Como exemplos, arquivos em disco, teclado, vídeo, impressora, portas de comunicação, etc.

Arquivos

- Assim, o conceito de arquivo é ampliado no sentido de considerar arquivos não somente os que existem em discos.
- Em C, podemos trabalhar com arquivos por meio de acesso em alto-nível ou em baixo-nível.

Arquivos

- O conjunto de funções que interagem com arquivos em alto-nível são bufferizadas, isto é, mantém uma área de memória para manipular os bytes antes de serem gravados ou para manipular bytes lidos antes de serem enviados ao programa.
- As funções de acesso em baixo-nível não são bufferizadas.

Roteiro

- Arquivos
- Leitura e gravação em alto-nível
- Abrindo arquivos com fopen()
- Gravar e ler um caractere por vez
- A função feof()
- Fim de arquivo e a constante EOF
- Cuidados ao abrir o arquivo
- Ler e gravar linha a linha
- Ler e gravar dados formatados
- Modos de abertura de arquivos
- Modo texto e modo binário
- Ler e gravar blocos de bytes
- Os ponteiros de arquivos stdin, stdout e stderr

Leitura e Gravação em Alto-Nível

- Para trabalharmos com um arquivo em disco, o nosso programa e o sistema operacional devem participar conjuntamente de certas informações sobre o arquivo.
- Em outras palavras, informações específicas sobre o arquivo devem estar presentes antes que o programa possa acessá-lo.
- Em C, as informações necessárias são guardadas em uma estrutura do tipo FILE.

Leitura e Gravação em Alto-Nível

- A definição dessa estrutura está contida no arquivo stdio.h.
- Se você examinar seu arquivo stdio.h, provavelmente encontrará algo semelhante ao mostrado a seguir:

*stdio.h

```
138  #define FOPEN_MAX    (20)
139
140  typedef struct _iobuf
141  {
142      char*    _ptr;
143      int _cnt;
144      char*    _base;
145      int _flag;
146      int _file;
147      int _charbuf;
148      int _bufsiz;
149      char*    _tmpfname;
150  } FILE;
```

Leitura e Gravação em Alto-Nível

- Cada arquivo necessita de informações similares, stdio.h declara uma matriz de estruturas FILE.
- A constante FOPEN_MAX determina a dimensão da matriz e representa o número máximo de arquivos que podem ser abertos simultaneamente por um programa.
- O número para FOPEN_MAX depende das considerações tomadas no projeto do compilador e das restrições do sistema operacional.

Leitura e Gravação em Alto-Nível

- Os membros da estrutura FILE contêm informações do arquivo, tais como: seu atual tamanho, a localização de seus buffers de dados, se o arquivo está sendo lido ou gravado etc.
- O programador não deve se preocupar com o conteúdo dos membros da estrutura FILE, e sim voltar seu interesse para a abertura de um arquivo e como trabalhar com ele.

Roteiro

- Arquivos
- Leitura e gravação em alto-nível
- Abrindo arquivos com fopen()
- Gravar e ler um caractere por vez
- A função feof()
- Fim de arquivo e a constante EOF
- Cuidados ao abrir o arquivo
- Ler e gravar linha a linha
- Ler e gravar dados formatados
- Modos de abertura de arquivos
- Modo texto e modo binário
- Ler e gravar blocos de bytes
- Os ponteiros de arquivos stdin, stdout e stderr

Abrindo Arquivos com fopen()

- A função fopen() executa duas tarefas.
- Em primeiro lugar, cria e preenche uma estrutura FILE com as informações necessárias para o programa e para o sistema operacional, de maneira que possam se comunicar.
- Em seguida, retorna um ponteiro do tipo FILE que aponta para a localização na memória dessa estrutura criada.

Abrindo Arquivos com fopen()

- Para trabalhar com arquivos em alto-nível, a linguagem C oferece um pacote de funções de biblioteca divididas em quatro grupos, a saber:
  - Grupo 1: Gravar e ler um caractere por vez – As funções fputc() e fgetc();
  - Grupo 2: Ler e gravar linha a linha – As funções fputs() e fgets();
  - Grupo 3: Ler e gravar dados formatados – As funções fprintf() e fscanf();
  - Grupo 4: Ler e gravar blocos de bytes – AS funções fwrite() e fread();

Roteiro

- Arquivos
- Leitura e gravação em alto-nível
- Abrindo arquivos com fopen()
- Gravar e ler um caractere por vez
- A função feof()
- Fim de arquivo e a constante EOF
- Cuidados ao abrir o arquivo
- Ler e gravar linha a linha
- Ler e gravar dados formatados
- Modos de abertura de arquivos
- Modo texto e modo binário
- Ler e gravar blocos de bytes
- Os ponteiros de arquivos stdin, stdout e stderr

Gravar e ler um caractere por vez

- As funções fputc() e fgetc() gravam e leem um caractere por vez.
- São análogas às funções putc() e getc(); a única diferença é que, ao contrario daquelas, estas têm implementação como macros.
- Nosso primeiro exemplo lê, do teclado, um caractere por vez e o grava no arquivo.

Gravar e ler um caractere por vez

Prog1.c

```c
/* Prog1.c */
/* Grava um caractere por vez num arquivo */
#include <stdio.h> /* define FILE */
#include <stdlib.h>
#include <conio.h>

int main(void)
{
    FILE *fptr;  /* ponteiro para arquivo */
    char ch;

    fptr=fopen("ArqText.txt","w");/*abre arq.p/gravar em modo texto */
    while((ch=getche()) != '\r')  /*Lê um caractere do teclado */
        fputc(ch,fptr);          /*grava caractere no arquivo */
    fclose(fptr);

    return 0;
}
```

Gravar e ler um caractere por vez

- Execute o programa e digite uma frase.
- A leitura do teclado termina quando você pressiona a tecla ENTER.
- A linha que você digitou será gravada no arquivo chamado arqtext.txt da pasta corrente.
- Para verificar se o arquivo foi realmente criado, use o Bloco de Notas do Windows para ler seu conteúdo.

Gravar e ler um caractere por vez

- Abrindo arquivos com fopen():
  - Para abrir um arquivo em alto-nível devemos usar a função fopen():

```
fptr=fopen("ArqText.txt","w");
```

  - Nosso programa cria o arquivo texto arqdest.txt para gravação (“w”).

Gravar e ler um caractere por vez

- Gravando no arquivo com fputc():
  - Uma vez estabelecida uma linha de comunicação com um arquivo particular por meio de sua abertura, podemos escrever nele.
  - A função fputc() recebe dois argumentos: o caractere a ser gravado e o ponteiro para a estrutura FILE do arquivo.
  - Ela retorna o caractere gravado ou EOF se acontecer algum erro:

```
fputc(ch,fptr);
```

Gravar e ler um caractere por vez

- Gravando no arquivo com fputc():
  - O processo de gravação continua a cada interação do laço while;
  - Toda vez que a função fputc() é executada, mais um caractere é gravado no arquivo.
  - Quando o usuário pressionar ENTER, o laço termina.

Gravar e ler um caractere por vez

- Fechando o arquivo com fclose():
  - Quando terminamos a gravação do arquivo, precisamos fechá-lo.

```
fclose(fptr);
```

  - Fechar um arquivo significa, em primeiro lugar, esvaziar o seu buffer, gravando nele os caracteres remanescentes.
  - Não falamos sobre buffer antes porque, quando se usa alto-nível, ele é invisível para o programador.

Gravar e ler um caractere por vez

- Fechando o arquivo com fclose():
  - Entretanto, o buffer é necessário, mesmo que invisível.
  - Considere, por exemplo, quão ineficiente seria acessar o disco somente para gravar um caractere.
  - Isto tomaria tempo para que o sistema de disco encontrasse a posição de gravação correta e, ainda, demoraria para que encontrasse o setor exato da trilha para cada caractere gravado.

Gravar e ler um caractere por vez

- Fechando o arquivo com fclose():
  - Se você digitar vários caracteres rapidamente e cada um necessitar de um acesso completo ao disco, vários deles serão, provavelmente, perdidos.
  - Eis como o buffer funciona: quando você envia um caractere para um arquivo com fputc(), ele é armazenado, temporariamente, numa área de memória reservada para essa tarefa.
  - Essa área de memória chama-se buffer.
  - Quando ele estiver lotado, seu conteúdo será gravado no arquivo de uma única vez.

Gravar e ler um caractere por vez

- Fechando o arquivo com fclose():
  - Uma das razões para fechar arquivos é que, nesse momento, o programa força a gravação do buffer nele.
  - Outra razão para fechar o arquivo é liberar as áreas de comunicação usadas, para que estejam disponíveis a outros arquivos.
  - Essas áreas incluem a estrutura FILE e o buffer.

Gravar e ler um caractere por vez

- Fechando o arquivo com fclose():
  - Outra função que fecha arquivos é a exit().
  - Ela difere da função fclose() em vários pontos.
  - Primeiro, exit() fecha todos os arquivos abertos.
  - Segundo, encerra o programa e devolve o controle ao sistema operacional.
  - A função fclose() simplesmente fecha o arquivo associado ao ponteiro FILE enviado como argumento.

Gravar e ler um caractere por vez

- Lendo o arquivo:
  - O próximo exemplo lê, do arquivo, um caractere por vez e o imprime no vídeo.

Prog2.c

```
1  /* Prog2.c */
2  /* Lê um caractere por vez de um arquivo */
3  #include <stdio.h> /* define FILE */
4  #include <stdlib.h>
5
6  int main(void)
7  {
8      FILE *fptr;  /* ponteiro para arquivo */
9      short int ch;
10
11     fptr = fopen("ArqText.txt","r");/*abre arq. p/ ler em modo texto*/
12
13     while((ch=fgetc(fptr)) != EOF)  /*Lê um caractere do arquivo */
14         printf("%c",ch);           /*imprime o caractere no vídeo */
15     fclose(fptr);
16
17     return 0;
18 }
```

Roteiro

- Arquivos
- Leitura e gravação em alto-nível
- Abrindo arquivos com fopen()
- Gravar e ler um caractere por vez
- A função feof()
- Fim de arquivo e a constante EOF
- Cuidados ao abrir o arquivo
- Ler e gravar linha a linha
- Ler e gravar dados formatados
- Modos de abertura de arquivos
- Modo texto e modo binário
- Ler e gravar blocos de bytes
- Os ponteiros de arquivos stdin, stdout e stderr

A Função feof()

- Essa função retorna verdadeiro se o final do arquivo tiver sido atingido; caso contrário, retorna falso:

```
int feof(FILE *fp);
```

- Como exemplo, observe a mudança no pedaço de código do programa anterior:

```
while(!feof(fptr))
{
    ch = fgetc(fptr);
    printf("%c",ch);
}
```

A Função feof()

- Abrindo arquivo para leitura:
  - Nosso programa abre o arquivo arqtext.txt para leitura (“r”).

```
fptr = fopen("ArqText.txt","r");
```

- Lendo qrquivo com fgetc():
  - A função fgetc() recebe como argumento o ponteiro para a estrutura FILE do arquivo.
  - Retorna o caractere lido ou EOF se encontrar o fim do arquivo.

```
fgetc(fptr)
```

Roteiro

- Arquivos
- Leitura e gravação em alto-nível
- Abrindo arquivos com fopen()
- Gravar e ler um caractere por vez
- A função feof()
- Fim de arquivo e a constante EOF
- Cuidados ao abrir o arquivo
- Ler e gravar linha a linha
- Ler e gravar dados formatados
- Modos de abertura de arquivos
- Modo texto e modo binário
- Ler e gravar blocos de bytes
- Os ponteiros de arquivos stdin, stdout e stderr

Fim de Arquivo e a Constante EOF

- A maior diferença o segundo programa e o primeiro é que este deve reconhecer quando o arquivo terminou.
- O que é EOF?
- É importante entender que EOF não é um caractere pertencente ao arquivo e sim uma constante, do tipo short int, definida no arquivo stdio.h com valor 0xFFFF ou -1.

Fim de Arquivo e a Constante EOF

- O fim de um arquivo é reconhecido pelo sistema operacional, o Windows envia 0xFFFF aos nossos programas para avisar que o arquivo acabou.
- Como podemos reconhecer que o arquivo terminou?
- Se utilizarmos uma variável do tipo char para receber bytes lidos do arquivo, o caractere de código ASCII 255 decimal (0xFF), se existir dentro do arquivo, será interpretado como EOF e a leitura terminará antecipadamente.

Fim de Arquivo e a Constante EOF

- Por causa disso criamos a variável ch do tipo short int.
- Dessa forma, não haverá confusão entre o caractere 0xFF e EOF.
- O primeiro será armazenado em ch como 0x00FF, e o segundo, como 0xFFFF.

Fim de Arquivo e a Constante EOF

- A mensagem enviada pelo sistema operacional indicando o fim de arquivo pode ser diferente para sistemas operacionais distintos.
- O arquivo stdio.h define EOF com o valor correto para o seu sistema operacional; assim, em seus programas, use essa constante para testar o fim de arquivo.

Fim de Arquivo e a Constante EOF

- Valer-se de um número mágico (por exemplo -1), em vez de EOF, pode causar problemas se o programa for transportado para outro sistema operacional.

Roteiro

- Arquivos
- Leitura e gravação em alto-nível
- Abrindo arquivos com fopen()
- Gravar e ler um caractere por vez
- A função feof()
- Fim de arquivo e a constante EOF
- Cuidados ao abrir o arquivo
- Ler e gravar linha a linha
- Ler e gravar dados formatados
- Modos de abertura de arquivos
- Modo texto e modo binário
- Ler e gravar blocos de bytes
- Os ponteiros de arquivos stdin, stdout e stderr

Cuidados Ao Abrir Arquivo

- Os dois programas que apresentamos têm uma falha.
- Se o arquivo especificado em fopen() não puder ser aberto, o programa irá parar.
- Por que um arquivo pode não ser aberto?
- Se você tiver pedido para abrir um arquivo para gravação, é provável que não abra por não haver mais espaço no disco.
- E se foi para leitura, talvez o arquivo não abra por ainda não ter sido criado.

Cuidados Ao Abrir Arquivo

- Então, é importante que todos os programas que acessem arquivos em disco verifiquem se o arquivo foi aberto com sucesso, antes de ler ou escrever neles.
- Se o arquivo não pôde ser aberto, a função fopen() retorna um ponteiro com o valor NULL.
- Vamos mostrar como alterar a instrução de abertura do arquivo para testar se foi aberto com sucesso.

Cuidados Ao Abrir Arquivo

Prog3.c

```c
/* Prog3.c */
/* Lê um caractere por vez de um arquivo */
#include <stdio.h> /* define FILE */
#include <stdlib.h>

int main(void)
{
    FILE *fptr;  /* ponteiro para arquivo */
    short int ch;

    if( (fptr = fopen("ArqText.txt","r")) == NULL)
    {
        puts("Não foi possível abrir o arquivo");
        exit(1);
    }

    while((ch=fgetc(fptr)) != EOF)  /*Lê um caractere do arquivo */
        printf("%c",ch);           /*imprime o caractere no vídeo */
    fclose(fptr);

    return 0;
}
```

Roteiro

- Arquivos
- Leitura e gravação em alto-nível
- Abrindo arquivos com fopen()
- Gravar e ler um caractere por vez
- A função feof()
- Fim de arquivo e a constante EOF
- Cuidados ao abrir o arquivo
- Ler e gravar linha a linha
- Ler e gravar dados formatados
- Modos de abertura de arquivos
- Modo texto e modo binário
- Ler e gravar blocos de bytes
- Os ponteiros de arquivos stdin, stdout e stderr

Ler e Gravar Linha a Linha

- As funções fputs() e fgets() gravam ou leem os dados linha a linha (como strings).
- O próximo exemplo cria um arquivo texto e grava nele uma string por vez.

Ler e Gravar Linha a Linha

Prog4.c

```c
/* Prog4.c */
/* Grava strings no arquivo */
#include <stdio.h> /* define FILE */
#include <stdlib.h>

int main(void)
{
    FILE *fptr;  /* ponteiro para arquivo */

    fptr = fopen("TesteSTR.txt","w");/*Cria p/gravar em modo texto */

    fputs("Um grande antídoto contra o egoísmo\n",fptr);
    fputs("é a generosidade... Dê, mesmo que\n", fptr);
    fputs("isso requeira de você um esforço\n", fptr);
    fputs("consciente. Pelo fato de partilhar\n",fptr);
    fputs("tudo o que possui, seu egoísmo se\n",fptr);
    fputs("abrandará.\n",fptr);
    fclose(fptr);
    system("TYPE TesteSTR.txt");
    system("PAUSE");
    return 0;
}
```

Ler e Gravar Linha a Linha

- Para verificar se o arquivo foi gravado corretamente, você pode utilizar algum processador de textos ou comando type do DOS na chamada da função system().

```
system("TYPE TesteSTR.txt");
```

Ler e Gravar Linha a Linha

- Gravando no arquivo com fputs():
  - A função fputs() recebe dois argumentos: o ponteiro char para a cadeia de caracteres a ser gravada, e o ponteiro para a estrutura FILE do arquivo.
  - Retorna um número positivo ou EOF se acontecer algum erro.

```
fputs("Um grande antídoto contra o egoísmo\n",fptr);
```

  - A função fputs() não coloca automaticamente o caractere de nova linha no fim dela.
  - Devemos adicioná-lo explicitamente.

Ler e Gravar Linha a Linha

- Lendo do arquivo:
  - O próximo exemplo lê o arquivo testestr.txt, criado pelo programa anterior, uma linha por vez.

Ler e Gravar Linha a Linha

Prog5.c

```c
/* Prog5.c */
/* Lê linha a linha do arquivo */
#include <stdio.h> /* define FILE */
#include <stdlib.h>

int main(void)
{
    FILE *fptr;  /* ponteiro para arquivo */
    char str[81];

    /*Abre p/leitura em modo texto*/
    if( (fptr = fopen(("TesteSTR.txt","r")) == NULL)
{
    puts("Não foi possível abrir o arquivo");
        exit(1);
    }

    while(fgets(str,80,fptr) != NULL) /*Lê uma linha de texto */
    printf("%s",str);

    fclose(fptr);
    system("pause");
    return 0;
}
```

Ler e Gravar Linha a Linha

- Lendo do arquivo com fgets():
  - A função fgets recebe três argumentos: o ponteiro char para a cadeia de caracteres em que os dados lidos do arquivo serão colocados, o número máximo de caracteres a serem lidos, e o ponteiro para a estrutura FILE do arquivo.
  - Retorna um ponteiro para a cadeia de caracteres lida ou NULL se encontrar algum erro ou o fim do arquivo.

```
fgets(str,80,fptr)
```

Roteiro

- Arquivos
- Leitura e gravação em alto-nível
- Abrindo arquivos com fopen()
- Gravar e ler um caractere por vez
- A função feof()
- Fim de arquivo e a constante EOF
- Cuidados ao abrir o arquivo
- Ler e gravar linha a linha
- Ler e gravar dados formatados
- Modos de abertura de arquivos
- Modo texto e modo binário
- Ler e gravar blocos de bytes
- Os ponteiros de arquivos stdin, stdout e stderr

Ler e Gravar Dados Formatados

- As funções fprintf() e fscanf() gravam ou leem os dados de modo formatado.
- Os exemplos anteriores mostraram como ler e gravar caracteres e textos em um arquivo.
- A pergunta é: como manusear dados numéricos em arquivos em disco?
- Para mostrar como trabalhar com dados numéricos, usaremos três itens relativos a livros de uma livraria: título (string), número de registro (inteiro) e preço (ponto flutuante)

Ler e Gravar Dados Formatados

Prog6.c

```
1  /* Prog6.c */
2  /* Grava dados formatados num arquivo */
3  #include <stdio.h> /* define FILE */
4  #include <stdlib.h>
5  #include <string.h>
6  #define TRUE 1
7
8  int main(void)
9  {
10     FILE *fptr; /* ponteiro para arquivo */
11     char titulo[30];
12     int regnum;
13     double preco;
14     fptr = fopen("Livros.txt","w");
15     while (TRUE)
16     {
17         printf("\nDigite titulo, registro e preco do livro: ");
18         scanf("%s %d %lf",titulo,&regnum,&preco);
19         if(strlen(titulo) <= 1) break;
20         fprintf(fptr,"%s %d %.2lf\n",titulo,regnum,preco);
21     }
22
23     fclose(fptr);
24     system("pause");
25     return 0;
26 }
```

Ler e Gravar Dados Formatados

- O ponto principal desse programa é a função fprintf(), que escreve os valores das três variáveis no arquivo.
- Essa função é similar a printf(), exceto pelo fato de que o ponteiro para file é tomado como primeiro argumento.
- Todas as possibilidades de formato de printf() operam com fprintf().
- As informações inseridas ao executar o código estão no arquivo livros.txt.

Ler e Gravar Dados Formatados

- Lendo do arquivo:

Prog7.c

```c
/* Prog7.c */
/* Lê dados formatados do arquivo */
#include <stdio.h> /* define FILE */
#include <stdlib.h>

int main(void)
{
    FILE *fptr; /* ponteiro para arquivo */
    char titulo[30];
    int regnum;
    double preco;
    fptr = fopen("Livros.txt","r");

    while ( fscanf(fptr,"%s %d %lf", titulo, &regnum, &preco) != EOF)
        printf("%s %d %.2lf\n", titulo, regnum, preco);

    fclose(fptr);
    system("pause");
    return 0;
}
```

Ler e Gravar Dados Formatados

- Lendo do arquivo:
  - O programa usa a função fscanf() para ler os dados do disco.
  - Essa função é similar à função scanf(), exceto pelo fato de que, como em printf(), um ponteiro para file deverá ser incluído como primeiro argumento.
  - O programa imprime os dados lidos com printf().

Roteiro

- Arquivos
- Leitura e gravação em alto-nível
- Abrindo arquivos com fopen()
- Gravar e ler um caractere por vez
- A função feof()
- Fim de arquivo e a constante EOF
- Cuidados ao abrir o arquivo
- Ler e gravar linha a linha
- Ler e gravar dados formatados
- Modos de abertura de arquivos
- Modo texto e modo binário
- Ler e gravar blocos de bytes
- Os ponteiros de arquivos stdin, stdout e stderr

Modos de Abertura de Arquivos

Tabela com os possíveis modos de abertura de arquivos:	
"r"	Abre um arquivo para leitura em modo texto. Se o arquivo não existir, a operação irá falhar e **fopen()** retornará NULL.
"w"	Cria um arquivo em modo texto para gravação. Se o arquivo já existir, elimina seu conteúdo e recomeça a gravação a partir de seu início.
"a"	Abre um arquivo em modo texto para gravação, a partir de seu final. Se o arquivo não existir, ele será criado.
"r+"	Abre um arquivo em modo texto para atualização, ou seja, tanto para leitura como para gravação. Se o arquivo não existir, a operação irá falhar e **fopen()** retornará NULL.
"w+"	Cria um arquivo em modo texto para atualização, ou seja, tanto para leitura como para gravação. Se o arquivo já existir, seu conteúdo será destruído.
"a+"	Abre um arquivo em modo texto para atualização, gravando novos dados a partir do final do arquivo. Se o arquivo não existir, ele será criado.
"rb"	Abre um arquivo para leitura em modo binário. Se o arquivo não existir, a operação irá falhar e **fopen()** retornará NULL.

Modos de Abertura de Arquivos

"rb"	Abre um arquivo para leitura em modo binário. Se o arquivo não existir, a operação irá falhar e **fopen()** retornará NULL.
"wb"	Cria um arquivo em modo binário para gravação. Se o arquivo já existir, elimina seu conteúdo e recomeça a gravação a partir de seu início.
"ab"	Abre um arquivo em modo binário para gravação, a partir de seu final. Se o arquivo não existir, será criado.
"rb+"	Abre um arquivo em modo binário para atualização, ou seja, tanto para leitura como para gravação. Se o arquivo não existir, a operação irá falhar e **fopen()** retornará NULL.
"wb+"	Cria um arquivo em modo binário para atualização, ou seja, tanto para leitura como para gravação. Se o arquivo já existir, seu conteúdo será destruído.
"ab+"	Abre um arquivo em modo binário para atualização, gravando novos dados a partir do final do arquivo. Se o arquivo não existir, ele será criado.

Roteiro

- Arquivos
- Leitura e gravação em alto-nível
- Abrindo arquivos com fopen()
- Gravar e ler um caractere por vez
- A função feof()
- Fim de arquivo e a constante EOF
- Cuidados ao abrir o arquivo
- Ler e gravar linha a linha
- Ler e gravar dados formatados
- Modos de abertura de arquivos
- Modo texto e modo binário
- Ler e gravar blocos de bytes
- Os ponteiros de arquivos stdin, stdout e stderr

Modo Texto e Modo Binário

- São três as diferenças de trabalhar com um arquivo em modo texto e em modo binário:
  - O caractere \n ou \r: Em uma operação de gravação em arquivos abertos em modo texto (padrão), o caractere \n ou \r é expandido em dois bytes, carriagereturn e linefeed (\r e \n), antes de ser gravado. Em operações de leitura, o par de bytes CR/LF é convertido para um único byte \n. Se o arquivo é aberto em modo binário, não há essa conversão.

Modo Texto e Modo Binário

- O caractere \x1A (Ctrl-Z): Em modo texto, é interpretado como fim do arquivo. Em modo binário, é mais um caractere pertencente ao arquivo.
- Números: Em modo binário podem ser gravados ou lidos exatamente como se apresentam na memória. Por exemplo, o número 25.678 é armazenado, na memória do computador, ocupando dois bytes (short int). Em modo texto, ocuparia 5 bytes e seria armazenado na forma ASCII.

Modo Texto e Modo Binário

- Para verificar essa diferença, vamos escrever um programa que conta os caracteres de um arquivo.

Modo Texto e Modo Binário

Prog8.c

```c
/*Prog8.c */
/*Conta caracteres do arquivo em modo texto */
#include <stdio.h> /* define FILE */
#include <stdlib.h>

int main(int argc,char **argv)
{
    FILE *fptr;  /* ponteiro para arquivo */
    short int ch;
    int cont=0;

    if(argc != 2)
    {
        puts("Forma de uso: C:\\>FileCont nomearq");
        exit(1);
    }

    if( (fptr = fopen(argv[1],"r")) == NULL)
    {
        puts("Não foi possível abrir o arquivo");
        exit(1);
    }

    while((ch=fgetc(fptr)) != EOF)  /*Lê um caractere do arquivo */
        cont++;                     /*incrementa contador de caracteres */
    printf("O tamanho do arquivo é %d bytes.\n", cont);
    fclose(fptr);

    return 0;
}
```

Modo Texto e Modo Binário

- Começamos verificando o número de argumentos da linha de comandos; se forem dois, então argv[1] será o nome do arquivo a ser aberto, caso contrário, o programa apresentará uma mensagem e terminará.
- O arquivo é aberto usando o modo texto (padrão).
- O laço while é executado, lê um caractere por vez e incrementa o contador de caracteres.

Modo Texto e Modo Binário

- Finalmente, o programa imprime o número de caracteres contados.
- Execute esse programa e depois use o comando DIR da console e compare o tamanho do arquivo apresentado pelo programa com o tamanho apresentado pelo sistema.
- A razão dessa diferença está no tratamento do caractere /n.

Modo Texto e Modo Binário

- Modificaremos o programa para que o arquivo seja aberto em modo binário:

Modo Texto e Modo Binário

Prog9.c

```c
/*Prog9.c */
/*Conta caracteres do arquivo em modo binário */
#include <stdio.h> /* define FILE */
#include <stdlib.h>

int main(int argc,char **argv)
{
    FILE *fptr;  /* ponteiro para arquivo */
    short int ch;
    int cont=0;

    if(argc != 2)
    {
        puts("Forma de uso: C:\\>FileCont nomearq");
        exit(1);
    }

    if( (fptr = fopen(argv[1],"rb")) == NULL)
    {
        puts("Não foi possível abrir o arquivo");
        exit(1);
    }

    while((ch=fgetc(fptr)) != EOF)  /*Lê um caractere do arquivo */
        cont++;           /*incrementa contador de caracteres */
    printf("O tamanho do arquivo é %d bytes.\n", cont);
    fclose(fptr);

    return 0;
}
```

Roteiro

- Arquivos
- Leitura e gravação em alto-nível
- Abrindo arquivos com fopen()
- Gravar e ler um caractere por vez
- A função feof()
- Fim de arquivo e a constante EOF
- Cuidados ao abrir o arquivo
- Ler e gravar linha a linha
- Ler e gravar dados formatados
- Modos de abertura de arquivos
- Modo texto e modo binário
- Ler e gravar blocos de bytes
- Os ponteiros de arquivos stdin, stdout e stderr

Ler e gravar blocos de bytes

- Muitas vezes não desejamos gravar ou ler um caractere ou uma linha por vez.
- Gostaríamos de gravar ou ler uma quantidade maior de caracteres.
- As funções apropriadas para o processo de gravação e leitura de matrizes de bytes ou estruturas são fwrite() e fread().

Ler e gravar blocos de bytes

Prog10.c

```c
/* Prog10.c */
/* Mostra o uso de fread() e fwrite() */
#include <stdio.h> /* define FILE */
#include <stdlib.h>

void CopiaArquivo(const char* Origem, const char *Destino);

int main()
{
    char Origem[] = "CopiaArq.C";
    char Destino[]= "CopiaArq.cpy";

    CopiaArquivo(Origem, Destino);
    system("type CopiaArq.cpy");
    system("pause");
    return 0;
}

```

Ler e gravar blocos de bytes

```
19  void CopiaArquivo(const char* Origem, const char *Destino)
20  {
21      unsigned char buffer[1024];
22      int BytesLidos;
23
24      FILE *fOrigem, *fDest; /* ponteiro para arquivo */
25
26      if((fOrigem = fopen(Origem,"rb"))==NULL) return;
27      if((fDest = fopen(Destino,"wb"))== NULL) return;
28      do
29      {
30          BytesLidos= fread(buffer,sizeof(char),sizeof(buffer), fOrigem);
31          fwrite(buffer,sizeof(char), BytesLidos, fDest);
32      }
33      while(BytesLidos);
34
35      fclose(fOrigem);
36      fclose(fDest);
37  }
```

Ler e gravar blocos de bytes

- Para ler um bloco de dados usamos fread()
- Em nosso exemplo queremos ler 1.024 bytes por vez:

```
BytesLidos= fread(buffer,sizeof(char),sizeof(buffer), fOrigem);
```

- A função retorna o número de unidades efetivamente lidas.
- Esse número pode ser menor que 1.024 quando o fim do arquivo for encontrado ou ocorrer um erro.

Ler e gravar blocos de bytes

- Para gravar blocos de bytes usamos fwrite().
- Em nosso exemplo queremos gravar BytesLidos a cada vez:

```
fwrite(buffer,sizeof(char), BytesLidos, fDest);
```

- A função que retorna o número de itens gravados. Esse valor será igual a BytesLidos, a menos que ocorra algum erro.
- Observe que os nossos arquivos foram abertos em modo binário. As funções fread() e fwrite() só trabalham corretamente em modo binário.

Roteiro

- Arquivos
- Leitura e gravação em alto-nível
- Abrindo arquivos com fopen()
- Gravar e ler um caractere por vez
- A função feof()
- Fim de arquivo e a constante EOF
- Cuidados ao abrir o arquivo
- Ler e gravar linha a linha
- Ler e gravar dados formatados
- Modos de abertura de arquivos
- Modo texto e modo binário
- Ler e gravar blocos de bytes
- Os ponteiros de arquivos stdin, stdout e stderr

Os ponteiros de arquivos stdin, stdout e stderr

- Se você analisar o seu arquivo stdio.h, encontrará as seguintes definições de constantes:

```
#define stdin  (&__iob_func()[0])
#define stdout (&__iob_func()[1])
#define stderr (&__iob_func()[2])
```

- São os três primeiros elementos da matriz de estruturas FILE.
- Essas constantes podem ser usadas para acessar qualquer um dos três arquivos ‘standard’ predefinidos pelo sistema operacional (leitura, gravação e saída de erro)

Os ponteiros de arquivos stdin, stdout e stderr

- Cada uma delas pode ser tratada como um ponteiro para uma estrutura FILE:

```
FILE *stdin;
FILE *stdout;
FILE *stderr;
```

- Por padrão, a leitura é do teclado, a gravação e a saída de erro são o vídeo.

Ponteiro	Stream
stdin	Entrada padrão (teclado)
stdout	Saída padrão (vídeo)
stderr	Saída de erro (vídeo)

Os ponteiros de arquivos stdin, stdout e stderr

- Esses ponteiros são constantes; seus valores não podem ser alterados, mas podem ser usados como argumentos de funções.
- Algumas funções, como getchar e putchar, usam stdin e stdout automaticamente.
- Você pode usar os ponteiros FILE definidos em stdio.h para acessar os arquivos de entrada e saída padrão. Como exemplo , a instrução:

Os ponteiros de arquivos stdin, stdout e stderr

```
fgets(string, 80, stdin);
```

lê uma cadeia de caracteres do teclado.
A instrução:

```
fputs(string, stdout);
```

imprimirá string no vídeo.

Referências

- MIZRAHI, Victorine V. **Treinamento em Linguagem C**, São Paulo: Pearson Prentice Hall, 2008;